

HUFFMAN CODING

Huffman Coding

- Huffman Coding is a way of encoding characters into a sequence of 0s and 1s.
- Huffman Coding can be applied to perform lossless data compression.
- Developed by David A. Huffman in 1952 when he was a PhD student at MIT.
- Huffman coding algorithm generates a set of codes based on the frequency of occurrence of the characters. When the data is coded using these characters, the resulting encoding has a shorter length compared to an encoding that uses, say 8 bits to encode the data.

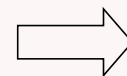
HUFFMAN CODES



Huffman Codes

- For compressing data (sequence of characters)
- Widely used
- Very efficient (saving 20-90%)
- Use a table to keep frequencies of occurrence of characters.
- Output binary string.

"Today's weather
is nice"



"001 0110 0 0 100
1000 1110"

HUFFMAN CODES

Example:

A file of 100,000 characters. Containing only 'a' to 'e'

	Frequency	Fixed-length codeword	Variable-length codeword
'a'	45000	000	0
'b'	13000	001	101
'c'	12000	010	100
'd'	16000	011	111
'e'	9000	100	1101
'f'	5000	101	1100

eg. "abc" = "000001010"

bits

eg. "abc" = "0101100"

$$\begin{aligned} &1 \cdot 45000 + 3 \cdot 13000 + 3 \cdot 12000 + \\ &3 \cdot 16000 + 4 \cdot 9000 + 4 \cdot 5000 \\ &= \text{bits} \end{aligned}$$



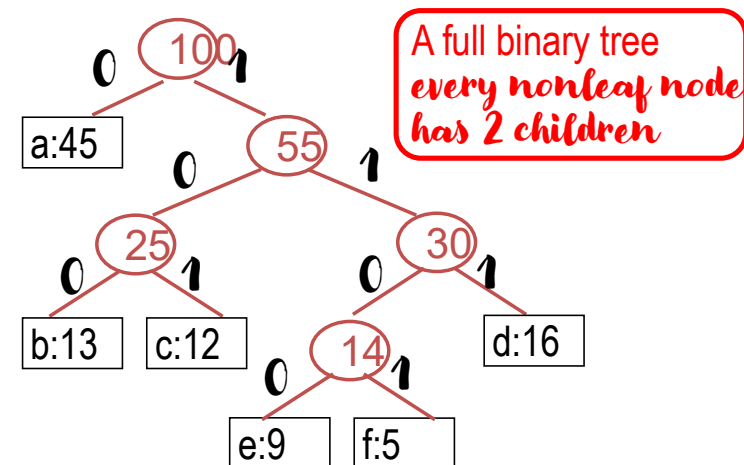
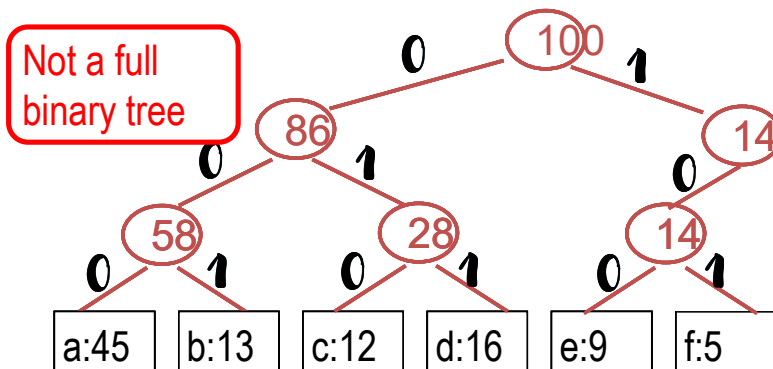
HUFFMAN CODES

A file of 100,000 characters.

The coding schemes can be represented by trees:

	Frequency (in thousands)	Fixed-length codeword
'a'	45	000
'b'	13	001
'c'	12	010
'd'	16	011
'e'	9	100
'f'	5	101

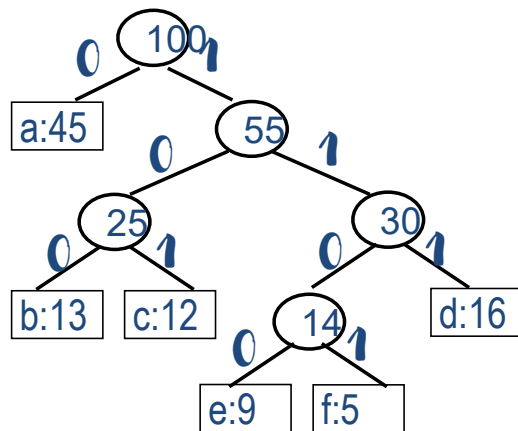
	Frequency (in thousands)	Variable-length codeword
'a'	45	0
'b'	13	101
'c'	12	100
'd'	16	111
'e'	9	1101
'f'	5	1100



HUFFMAN CODES



	Frequency	Codeword
'a'	45000	0
'b'	13000	101
'c'	12000	100
'd'	16000	111
'e'	9000	1101
'f'	5000	1100



Eg. "abc" is coded as "0101100"

To find an **optimal code** for a file:

1. The coding must be **unambiguous**.

Consider codes in which no codeword is also a prefix of other codeword. => **Prefix Codes**

Prefix Codes are unambiguous.

Once the codewords are decided, it is easy to compress (encode) and decompress (decode).

2. File size must be **smallest**.

=> Can be represented by a full binary tree.

=> Usually less frequent characters are at bottom

Let C be the alphabet (eg. $C = \{'a', 'b', 'c', 'd', 'e', 'f'\}$)

For each character c, no. of bits to encode all c's occurrences = $\text{freq}_c * \text{depth}_c$

File size $B(T) = \sum_{c \in C} \text{freq}_c * \text{depth}_c$

HUFFMAN CODES

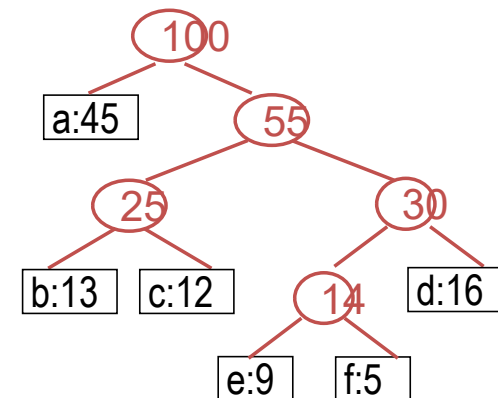
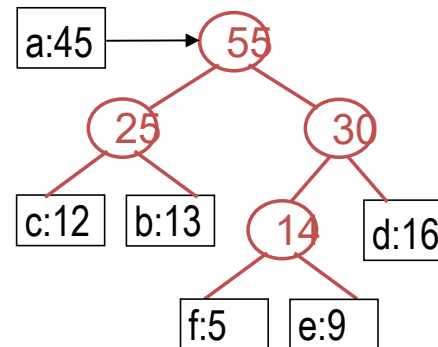
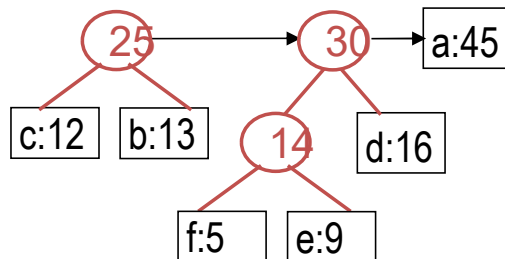
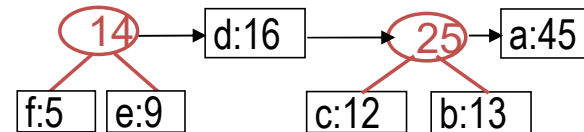
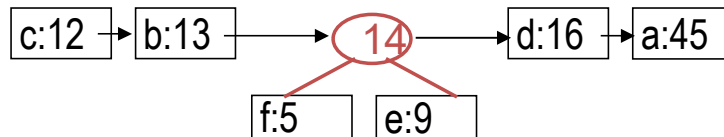


How do we find the optimal prefix code?

Huffman code (1952) was invented to solve it.
A Greedy Approach.

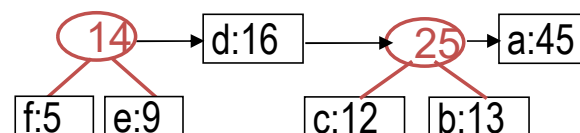
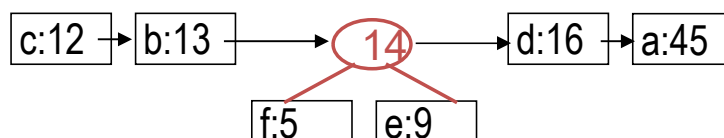
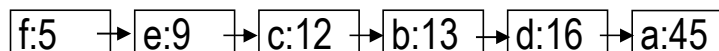


Q: A min-priority queue



HUFFMAN CODES

Q: A min-priority queue



....

HUFFMAN(C)

- 1 Build Q from C
- 2 For $i = 1$ to $|C|-1$
- 3 Allocate a new node z
- 4 $z.\text{left} = x = \text{EXTRACT_MIN}(Q)$
- 5 $z.\text{right} = y = \text{EXTRACT_MIN}(Q)$
- 6 $z.\text{freq} = x.\text{freq} + y.\text{freq}$
- 7 Insert z into Q in correct position.
- 8 Return $\text{EXTRACT_MIN}(Q)$

If Q is implemented as a binary min-heap,

"Build Q from C " is $O(n)$

"EXTRACT_MIN(Q)" is $O(\lg n)$

"Insert z into Q " is $O(\lg n)$

Huffman(C) is $O(n \lg n)$

How is it "greedy"?

CONSTRUCTING A HUFFMAN CODE

HUFFMAN(C) // C is a set of n characters

1 $n \leftarrow |C|$

2 $Q \leftarrow C$ // Q is implemented as a binary min-heap $O(\lg n)$

3 for $i \leftarrow 1$ to $n - 1$

Total computation time = $O(n \lg n)$

4 do allocate a new node z

5 $left[z] \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$ $O(\lg n)$

6 $right[z] \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$ $O(\lg n)$

7 $f[z] \leftarrow f[x] + f[y]$

8 $\text{INSERT}(Q, z)$ $O(\lg n)$

9 return $\text{EXTRACT-MIN}(Q)$

▷ Return the root of the tree.

COST OF A TREE T

- For each character c in the alphabet C
 - let $f(c)$ be the frequency of c in the file
 - let $d_T(c)$ be the depth of c in the tree
 - It is also the length of the codeword. Why?
- Let $B(T)$ be the number of bits required to encode the file (called the cost of T)

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

HUFFMAN CODE PROBLEM

In the pseudocode that follows:

We assume that C is a set of n characters and that each character $c \in C$ is an object with a defined frequency $f[c]$.

The algorithm builds the tree T corresponding to the optimal code

A min-priority queue Q , is used to identify the two least-frequent objects to merge together.

The result of the merger of two objects is a new object whose frequency is the sum of the frequencies of the two objects that were merged.

RUNNING TIME OF HUFFMAN'S ALGORITHM

The running time of Huffman's algorithm assumes that Q is implemented as a binary min-heap.

For a set C of n characters, the initialization of Q in line 2 can be performed in $O(n)$ time using the BUILD-MINHEAP

The for loop in lines 3–8 is executed exactly $n - 1$ times, and since each heap operation requires time $O(\lg n)$, the loop contributes $O(n \lg n)$ to the running time. Thus, the total running time of HUFFMAN on a set of n characters is $O(n \lg n)$.

PREFIX CODE

- Prefix(-free) code: no codeword is also a prefix of some other codewords (Un-ambiguous)
 - An optimal data compression achievable by a character code can always be achieved with a prefix code
 - Simplify the encoding (compression) and decoding
 - Encoding: $abc \rightarrow 0, 101, 100 = 0101100$
 - Decoding: $001011101 = 0, 0, 101, 1101 \rightarrow aabe$
 - Use binary tree to represent prefix codes for easy decoding
- An optimal code is always represented by a **full binary tree**, in which every non-leaf node has two children
 - $|C|$ leaves and $|C|-1$ internal nodes Cost:

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

Diagram illustrating the components of the formula:

- $f(c)$ is labeled "Frequency of c".
- $d_T(c)$ is labeled "Depth of c (length of the codeword)".

Huffman Codes are Optimal

Lemma: Consider the two letters, x and y with the smallest frequencies. Then is an optimal code tree in which these two letters are sibling leaves in the tree in the lowest level.

Proof: Let T be an optimum prefix code tree, and let b and c be two siblings at the maximum depth of the tree (must exist because T is full). Assume without loss of generality that $f(b) \leq f(c)$ and $f(x) \leq f(y)$ (if this is not true, then rename these characters). Since x and y have the two smallest frequencies it follows that $f(x) \leq f(b)$ (they may be equal) and $f(y) \leq f(c)$ (may be equal). Because b and c are at the deepest level of the tree we know that $d(b) \geq d(x)$ and $d(c) \geq d(y)$.

Now switch the positions of x and b in the tree resulting in a different tree T' and see how the cost changes. Since T is optimum,

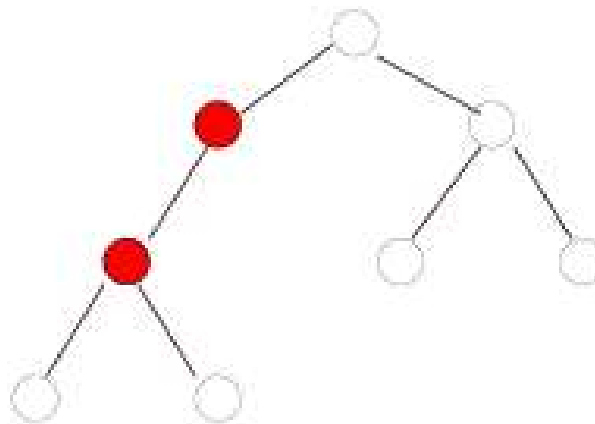
$$\begin{aligned} B(T) &\leq B(T') \\ &= B(T) - f(x)d(x) - f(b)d(b) + f(x)d(b) + f(b)d(x) \\ &= B(T) - (f(b) - f(x))(d(b) - d(x)) \\ &\leq B(T). \end{aligned}$$

Therefore, $B(T') = B(T)$, that is, T' is an optimum tree. By switching y with c we get a new tree T'' which by a similar argument is optimum. The final tree T'' satisfies the statement of the claim.

An Observation: Full Trees

Lemma: The tree for any **optimal prefix code** must be “**full**”, meaning that every internal node has exactly two children.

Proof: If some internal node had only one child then we could simply get rid of this node and replace it with its unique child. This would decrease the total cost of the encoding.



Huffman Codes are Optimal

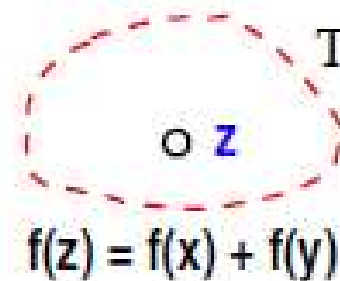
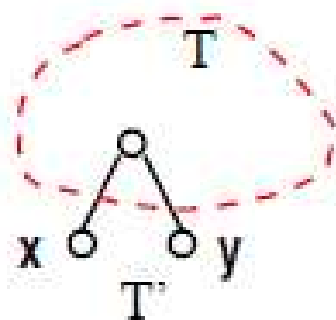
Theorem: Huffman's algorithm produces an optimum prefix code tree.

Proof: By induction on n . When $n = 2$, obvious.

Assume inductively that with strictly fewer than n letters, Huffman's algorithm is guaranteed to produce an optimum tree. We want to show this is also true with exactly n letters.

Huffman Codes are Optimal

Proof – continued: Consider an alphabet A' of n letters. Let T' be an optimum tree for A' with the two letters of lowest frequency x and y as sibling leaves (exists by Lemma). Let T be the coding tree for $A = A' \cup \{z\} - \{x, y\}$ ($n - 1$ leaves) obtained by removing x and y and replacing their parent by z .



$d(x) = d(y) = d(z) - 1$ and $f(z) = f(x) + f(y)$,

so

$$\begin{aligned} B(T) &= B(T') - f(x)d(x) - f(y)d(y) + f(z)d(z) \\ &= B(T') - (f(x) + f(y)). \end{aligned}$$

Huffman Codes are Optimal

Proof – continued: By the induction hypotheses, the Huffman algorithm gives a Huffman code tree H_A that is optimal for A . Let $H_{A'}$ be the tree obtained by adding x and y as children of z in H_A . Note that, as in the calculations for T and T' , we have:

$$B(H_A) = B(H_{A'}) - (f(x) + f(y)).$$

Also, note that, by the optimality of H_A , we have that $B(H_A) \leq B(T)$, so

$$\begin{aligned} B(H_{A'}) &= B(H_A) + f(x) + f(y) \\ &\leq B(T) + f(x) + f(y) \\ &= B(T'). \end{aligned}$$

Since we started with the assumption that T' was an optimal tree for A' this implies that $H_{A'}$, which is exactly the tree constructed by the Huffman algorithm, is also an optimal tree for A' and we are done.

HUFFMAN CODE

- Reduce size of data by 20%-90% in general
- If no characters occur more frequently than others, then no advantage over ASCII
- Encoding:
 - Given the characters and their frequencies, perform the algorithm and generate a code. Write the characters using the code
- Decoding:
 - Given the Huffman tree, figure out what each character is (possible because of prefix property)

APPLICATION ON HUFFMAN CODE

- Both the .mp3 and .jpg file formats use Huffman coding at one stage of the compression